# SE EXPERIMET -3
# Batch – A3
# Team Details:

# Stavan Desai (UID: 202430042)
# Dev Gaglani (UID: 2024300057)


**Aim:**
To draw Class diagram for Inventory and Stock Management System

## Implementation:

Mentioned below is the chronological order that was followed to design a Class diagram for our Inventory and Stock Management System. Initially, the Software Requirements Specification (SRS) and problem description were re-visited to identify key nouns and noun phrases. These were then reviewed to get rid of any ambiguity and the core data classes were identified. Relations between different classes, inheritance structures (such as different user roles generalizing from a base User class), and how they interact with each other were noted to get an idea about the overall structure of the class diagram. Finally, the class diagram was constructed keeping the above-mentioned points in mind and mapped to the specific attributes and functions required by the system.

## Problem Description:

An Inventory and Stock Management System needs to be designed to track, control, and optimize inventory across multiple warehouses and omnichannel retail endpoints. This software should be capable of authorizing different types of users and allowing them to perform their respective specialized functions based on Role-Based Access Control.

Administrators must be able to manage user roles and system-wide configurations. Managers are responsible for overseeing supplier relationships, tracking vendor performance, and approving or rejecting purchase orders (POs). Warehouse Staff must be able to log the physical receipt of goods to increase stock and utilize a voice-command interface to update inventory levels seamlessly on the warehouse floor. Sales Personnel must be able to log physical store sales and process returns, while the system automatically captures digital sales and processes webhooks from online marketplaces.

The system must maintain real-time stock records for every Product/SKU, automatically deduct stock upon order confirmation, and generate invoices. Furthermore, the software must incorporate an automated forecasting module to analyse historical data, predict future demand, recommend reorder points, and

trigger multimodal notifications (audio, email, SMS) to alert the relevant personnel about low stock levels or shipping delays.

**[1] Noun/ Noun Phrases**

Inventory, Stock, Administrator, Manager, Warehouse Staff, Sales Personnel, System, Product, SKU, Stock Record, Bin, Location, Omnichannel, Supplier, Vendor, Purchase Order, Sales Order, Invoice, Marketplace, Webhook, Speech-to-Text Engine, ML Forecasting Service, Forecast, Reorder Point, Seasonality, Notification, Alert, Customer, Payment.

## [2] Classes

User, Administrator, Manager, WarehouseStaff, SalesPersonnel, Product, StockRecord, Supplier, PurchaseOrder, SalesOrder, Forecast, Notification.

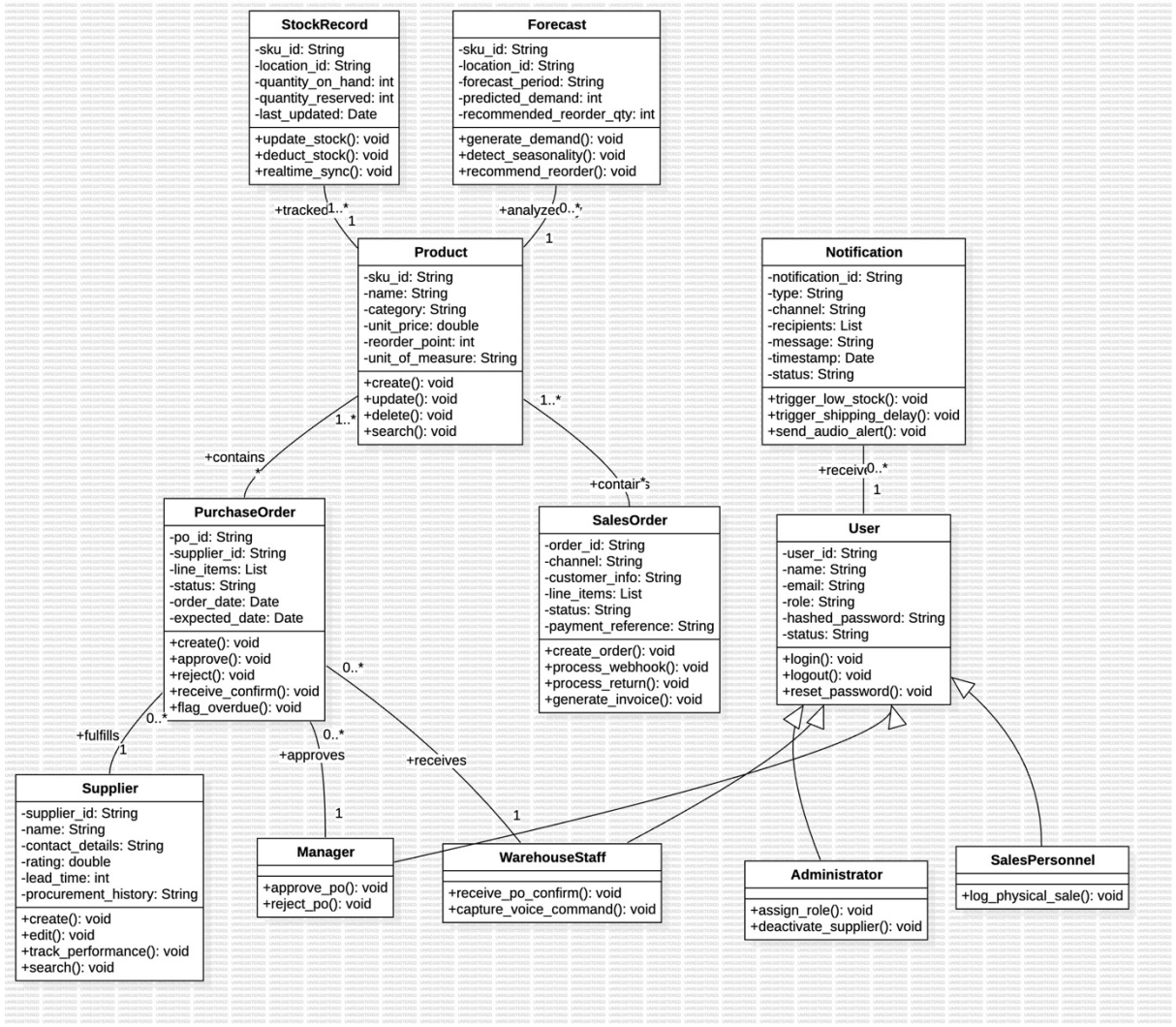
## [3] Verb Phrases

1. User logs into the system securely.
2. Administrator assigns role permissions and manages users.
3. Manager adds new suppliers and tracks their performance history.
4. Manager creates, approves, or rejects Purchase Orders.
5. Warehouse Staff confirms receipt of Purchase Orders to increase stock.
6. Warehouse Staff uses voice commands to adjust stock quantities.
7. Sales Personnel logs physical sales and processes customer returns.
8. System automatically processes marketplace webhooks for digital sales.
9. System deducts stock and generates invoices upon order confirmation.
10. System generates demand forecasts and recommends reorder points.
11. System triggers multimodal notifications (audio, SMS, email) for low stock or shipping delays.

## [4] Relations

1. Administrator, Manager, Warehouse Staff, and Sales Personnel are different types of users of the software and hence are generalized to **User**.
2. A **Product/SKU** can have multiple **Stock Records** tracked across different warehouses or store locations.
3. A **Product/SKU** can be associated with multiple **Forecast** records for different locations or time periods.
4. A **Supplier** fulfils zero or more **Purchase Orders**.
5. A **Purchase Order** contains one or more **Products** as line items.
6. A **Sales Order** contains one or more **Products** as line items.
7. A **User** can receive zero or more **Notifications**.
8. A **Manager** approves zero or more **Purchase Orders**.
9. **Warehouse Staff** receives and confirms zero or more **Purchase Orders**.

## [5] Class Diagram




## [5] Class Diagram Code Snippets

### User

```java
public class User{
    private String user_id;
    private String name;
    private String email;
    private String role;
    private String hashed_password;
    private String status;

    public void login(){}
    public void logout(){}
    public void reset_password(){}
}
```

**Administrator**

```
public class Administrator extends User{

    public void assign_role(){}
    public void deactivate_supplier(){}
}
```

**Manager**

```
public class Manager extends User{

    public void approve_po(){}
    public void reject_po(){}
}
```

**Warehouse Staff**

```
public class WarehouseStaff extends User{

    public void receive_po_confirm(){}
    public void capture_voice_command(){}
}
```

**Sales Personal**

```
public class SalesPersonnel extends User{

    public void log_physical_sale(){}
}
```

**Product**

```
public class Product{
    private String sku_id;
    private String name;
    private String category;
    private double unit_price;
    private int reorder_point;
    private String unit_of_measure;

    public void create(){}
    public void update(){}
    public void delete(){}
    public void search(){}
}
```

**Stock Record**

```
public class StockRecord{
```

```
    private String sku_id;
    private String location_id;
    private int quantity_on_hand;
    private int quantity_reserved;
    private Date last_updated;

    public void update_stock(){}
    public void deduct_stock(){}
    public void realtime_sync(){}
}
```

**Forecast**

```
public class Forecast{
    private String sku_id;
    private String location_id;
    private String forecast_period;
    private int predicted_demand;
    private int recommended_reorder_qty;

    public void generate_demand(){}
    public void detect_seasonality(){}
    public void recommend_reorder(){}
}
```

**Supplier**

```
public class Supplier{
    private String supplier_id;
    private String name;
    private String contact_details;
    private double rating;
    private int lead_time;
    private String procurement_history;

    public void create(){}
    public void edit(){}
    public void track_performance(){}
    public void search(){}
}
```

**Purchase Order**

```
public class PurchaseOrder{
    private String po_id;
    private String supplier_id;
    private List line_items;
    private String status;
    private Date order_date;
    private Date expected_date;
```

```
    public void create(){}
    public void approve(){}
    public void reject(){}
    public void receive_confirm(){}
    public void flag_overdue(){}
}
```

**Sales Order**

```
public class SalesOrder{
    private String order_id;
    private String channel;
    private String customer_info;
    private List line_items;
    private String status;
    private String payment_reference;

    public void create_order(){}
    public void process_webhook(){}
    public void process_return(){}
    public void generate_invoice(){}
}
```

**Notification**

```
public class Notification{
    private String notification_id;
    private String type;
    private String channel;
    private List recipients;
    private String message;
    private Date timestamp;
    private String status;

    public void trigger_low_stock(){}
    public void trigger_shipping_delay(){}
    public void send_audio_alert(){}
}
```

**Conclusion:**

By performing this experiment, we understood the concepts involved in designing a UML class diagram for an Inventory and Stock Management System. We identified the major classes, their attributes, methods, relationships, and inheritance structures such as the different user roles derived from the User class. We also understood how associations between products, stock records, forecasts, suppliers, purchase orders, sales orders, and notifications represent the interactions within the system. Finally, we mapped the classes and their operations to corresponding Java class declarations, providing a better understanding of how UML class diagrams can be translated into an object-oriented software design.